# Shared blocks — create once per crane (identical everywhere)

These four blocks are the **proven Line 4 program** (`docs/line4/instructions.md`). They are the **same for every crane** — you never edit the logic; you only **duplicate** the DB + FB per crane and change the DB name they reference. Create them in TIA exactly as the Line 4 runbook describes.

Per crane `<ID>` (e.g. `6`, `7A`, `1A`) you end up with:

- a data block **`Crane<ID>Data`** (copy of the DB below),
- a function block **`Crane<ID>_Seq`** (copy of `CraneSequencer` with `Line4Data` → `Crane<ID>Data`),
- shared **`typeTask`** UDT and **`TravelTo`** FC (create once for the whole PLC).

---

## 1) UDT `typeTask` — one task row (create once per PLC)

Project tree → **PLC data types** → **Add new data type** → name `typeTask`:

| Name | Data type | Start | Purpose |
|---|---|---|---|
| `station` | `Int` | `0` | target station index (drives motion) |
| `dwell_s` | `Int` | `0` | dip/dwell seconds in the tank |
| `crane` | `String[4]` | `''` | display, e.g. `7A` |
| `tank_name` | `String[10]` | `''` | display, e.g. `1-1A` |
| `prev` | `Int` | `0` | previous tank (traceability) |
| `next` | `Int` | `0` | next tank (traceability) |
| `status` | `Int` | `0` | 0 pending / 1 running / 2 done |

---

## 2) DB `Crane<ID>Data` — the OPC UA interface (copy per crane)

Add new block → **Data block (DB)** → **Global**, **Optimized access ON**. Name it for the crane (`Crane6Data`, `Crane7AData`, …). Rows:

**Batch**

| Name | Data type |
|---|---|
| `task_count` | `Int` |
| `tasks` | `Array[1..20] of "typeTask"` |

**Handshake** (`cmd_send` W; the rest R)

| Name | Data type |
|---|---|
| cmd_send | Bool |
| sts_busy | Bool |
| sts_done | Bool |
| sts_step | Int |
| sts_state | Int |

**I/O mirror** (mapped to real %I/%Q by the PLC's IOMap FC)

| Name | Data type | Notes |
|---|---|---|
| ls_at | Array[0..20] of Bool | station limit switches (ls_at[s] = crane over station s) |
| ls_dip | Bool | hoist fully lowered (in tank/loading) |
| ls_up | Bool | hoist fully raised |
| ls_mid | Bool | hoist at mid (optional; mirror + HMI) |
| btn_start | Bool | operator start/hold button |
| mv_fwd | Bool | travel toward higher station (East) |
| mv_rev | Bool | travel toward lower station (West) |
| mv_up | Bool | hoist up |
| mv_down | Bool | hoist down |
| estop | Bool | E-Stop input mirror (interlock = follow-up) |
| auto | Bool | Auto-mode input mirror |

> ls_at is [0..20] (Line 4 max station is 18; other lines up to 20). Only the crane's own tank indices are wired — the rest stay FALSE and are never targeted (the app sends a crane only its own tanks). **Tick *Accessible + Writable from HMI/OPC UA*** on every app-written row (task_count, cmd_send, tasks[].station/.dwell_s); the sts_*/status rows need only *Accessible*. ls_mid/estop/auto are mirrored for the IOMap + HMI; the proven FB below does not gate on them yet (safety interlocks are a deliberate follow-up).

---

## 3) FC TravelTo — direction picker (create once per PLC, SCL)

Interface — **Input:** target:Int, cur:Int, atTarget:Bool; **Output:** fwd:Bool, rev:Bool; **Return:** TravelTo:Bool. Body:

```
// TravelTo – pick the travel direction toward a target station.
// Returns TRUE when the target limit switch is already made (arrived → stop).
#fwd := FALSE;
#rev := FALSE;
```

```
IF #atTarget THEN
    ;                       // arrived: leave both motors off
ELSIF #target > #cur THEN
    #fwd := TRUE;           // higher station number → travel forward (East)
ELSE
    #rev := TRUE;           // lower station number → travel reverse (West)
END_IF;

#TravelTo := #atTarget;
```

---

## 4) FB `Crane<ID>_Seq` — the sequencer (copy per crane, SCL)

Add new block → **Function block (FB)**, **SCL**, name `Crane<ID>_Seq` (e.g. `Crane6_Seq`). **Static** members: `state:Int`, `idx:Int`, `count:Int`, `cur:Int`, `prevSend:Bool`, `tHold:TON_TIME`, `tDwell:TON_TIME`. **Temp:** `target:Int`, `arrived:Bool`.

Paste the body below, then **find/replace** `Line4Data` → `Crane<ID>Data` (the ONLY change per crane). OB1 calls each crane's FB once per scan (its instance DB is created automatically).

```
// Crane<ID>_Seq — walk the Crane<ID>Data.tasks[] batch through the crane
sequence.
// Call once per scan from OB1. All shared data lives in global DB "Line4Data"
(rename per crane).
//
// Station map: 0=Parking  1=Loading  2..N = tanks (tank number + 1)
// States: 0 IDLE · 10 GO_LOAD · 20 LOWER_LOAD · 30 WAIT_BTN · 40 LIFT_LOAD
//         50 GO_TANK · 60 LOWER_TANK · 70 DWELL · 80 LIFT_TANK · 85 GO_PARK · 90
DONE

"Line4Data".sts_state := #state;

CASE #state OF
    0:  // IDLE — parked, hoist up, motors off, waiting for a batch.
        "Line4Data".mv_fwd  := FALSE;  "Line4Data".mv_rev  := FALSE;
        "Line4Data".mv_up   := FALSE;  "Line4Data".mv_down := FALSE;
        "Line4Data".sts_busy := FALSE;
        "Line4Data".sts_step := 0;
        IF "Line4Data".cmd_send AND NOT #prevSend AND "Line4Data".task_count > 0
THEN
            #count := "Line4Data".task_count;
            #idx   := 1;
            "Line4Data".sts_done := FALSE;
            "Line4Data".sts_busy := TRUE;
            #state := 10;
        END_IF;

    10: // GO_LOAD — travel to the Loading Area (station 1).
        "Line4Data".sts_busy := TRUE;
        #target  := 1;
```

```
        #arrived := "TravelTo"(target := #target, cur := #cur,
                               atTarget := "Line4Data".ls_at[#target],
                               fwd => "Line4Data".mv_fwd, rev =>
"Line4Data".mv_rev);
        IF #arrived THEN
            #cur := #target;
            "Line4Data".mv_fwd := FALSE; "Line4Data".mv_rev := FALSE;
            #state := 20;
        END_IF;

    20: // LOWER_LOAD — lower the hoist until the dip limit switch is made.
        "Line4Data".mv_up := FALSE;  "Line4Data".mv_down := TRUE;
        IF "Line4Data".ls_dip THEN
            "Line4Data".mv_down := FALSE;
            #state := 30;
        END_IF;

    30: // WAIT_BTN — operator loads material, then HOLDS btn_start for 3 s.
        #tHold(IN := "Line4Data".btn_start, PT := T#3S);
        IF #tHold.Q THEN
            #state := 40;
        END_IF;

    40: // LIFT_LOAD — raise the hoist clear before travelling.
        "Line4Data".mv_down := FALSE; "Line4Data".mv_up := TRUE;
        IF "Line4Data".ls_up THEN
            "Line4Data".mv_up := FALSE;
            #state := 50;
        END_IF;

    50: // GO_TANK — travel to THIS task's tank station (fwd or rev; TravelTo
reverses if lower).
        "Line4Data".sts_step := #idx;
        "Line4Data".tasks[#idx].status := 1;                  // running
        #target  := "Line4Data".tasks[#idx].station;
        #arrived := "TravelTo"(target := #target, cur := #cur,
                               atTarget := "Line4Data".ls_at[#target],
                               fwd => "Line4Data".mv_fwd, rev =>
"Line4Data".mv_rev);
        IF #arrived THEN
            #cur := #target;
            "Line4Data".mv_fwd := FALSE; "Line4Data".mv_rev := FALSE;
            #state := 60;
        END_IF;

    60: // LOWER_TANK — dip into the tank until the dip limit switch is made.
        "Line4Data".mv_up := FALSE; "Line4Data".mv_down := TRUE;
        IF "Line4Data".ls_dip THEN
            "Line4Data".mv_down := FALSE;
            #state := 70;
        END_IF;

    70: // DWELL — hold in the tank for this task's dwell_s seconds.
        #tDwell(IN := TRUE,
```

```
                PT := DINT_TO_TIME(INT_TO_DINT("Line4Data".tasks[#idx].dwell_s) *
1000));
        IF #tDwell.Q THEN
            #tDwell(IN := FALSE);            // reset the timer for the next tank
            #state := 80;
        END_IF;

    80: // LIFT_TANK — lift out, mark the task done, advance or finish.
        "Line4Data".mv_down := FALSE; "Line4Data".mv_up := TRUE;
        IF "Line4Data".ls_up THEN
            "Line4Data".mv_up := FALSE;
            "Line4Data".tasks[#idx].status := 2;             // done
            IF #idx < #count THEN
                #idx := #idx + 1;
                #state := 50;                                // next tank
            ELSE
                #state := 85;                                // all tanks done
            END_IF;
        END_IF;

    85: // GO_PARK — return home to Parking (station 0).
        #target  := 0;
        #arrived := "TravelTo"(target := #target, cur := #cur,
                               atTarget := "Line4Data".ls_at[#target],
                               fwd => "Line4Data".mv_fwd, rev =>
"Line4Data".mv_rev);
        IF #arrived THEN
            #cur := #target;
            "Line4Data".mv_fwd := FALSE; "Line4Data".mv_rev := FALSE;
            #state := 90;
        END_IF;

    90: // DONE — raise the done flag, wait for the app to acknowledge.
        "Line4Data".sts_busy := FALSE;
        "Line4Data".sts_done := TRUE;
        IF NOT "Line4Data".cmd_send THEN
            "Line4Data".sts_done := FALSE;
            "Line4Data".sts_step := 0;
            #state := 0;
        END_IF;
END_CASE;

#prevSend := "Line4Data".cmd_send;   // edge memory (runs every scan, after the
CASE)
```

> **Downstream cranes (fed by a transfer tank, e.g. crane 6, 7B):** they have **no Loading bay** — they pick material up **at their transfer tank**. See the per-PLC README for how `ls_at[1]` (Loading) is mapped to the transfer-tank switch so states 10–40 still complete. The two-crane handoff timing is a plant-specific follow-up.